Verification par e-mail

Inscription StudentHub, codes de confirmation et service Resend

Ce qui est implemente : l'application demande un code a six chiffres a l'inscription, le verifie, conserve l'etat dans SQLite et affiche le badge **Verifie** sur le profil. Les comptes valides attendent ensuite l'approbation de l'administrateur.

Parcours dans l'application

1 Inscription	Le compte est cree dans la base SQLite de l'appareil avec l'e-mail non verifie et le statut en attente.
2 Code e-mail	Le serveur Node envoie un code avec Resend. Le code expire apres 10 minutes et les essais sont limites.
3 Confirmation	L'application confirme le code au serveur puis marque l'adresse comme verifiee dans SQLite.
4 Approbation admin	L'administrateur valide ou bloque les comptes locaux. Seuls les comptes verifies et valides peuvent se connecter.

Architecture actuelle

Flutter appelle les routes du service Node a l'adresse definie par `ACCOUNT_API_URL`. Le serveur utilise la cle Resend, conserve les codes temporairement en memoire et appelle l'API Resend. La cle ne doit jamais etre ajoutee au code Flutter.

SQLite reste local a chaque appareil. L'ecran Gestion des comptes ne voit que les comptes presents dans la base de l'appareil administrateur. Pour une vraie administration partagee entre utilisateurs, il faudra une base serveur commune.

Configurer Resend et lancer

Guide de mise en route pour l'envoi des codes de verification

1. Compte et expediteur Resend

Creez une cle API dans Resend. Ajoutez un domaine d'envoi que vous possedez et terminez sa verification DNS, puis utilisez une adresse sur ce domaine comme expediteur. La cle API est secrete et doit rester cote serveur.

> Pour un essai, `onboarding@resend.dev` est reserve a l'envoi vers l'adresse associee au compte Resend. Pour les inscriptions d'autres personnes, configurez un domaine verifie dans Resend.

2. Variables privees du serveur

Copiez le modele `backend/.env.example` vers `backend/.env`, puis completez les valeurs. Le fichier prive est ignore par Git.

```
PORT=6500
RESEND_API_KEY=votre_cle_resend_complete
RESEND_SENDER_EMAIL=welcome@votre-domaine-verifie.com
RESEND_SENDER_NAME=StudentHub
```

Dans PowerShell, depuis la racine StudentHub, demarrez le service et laissez ce terminal ouvert :

```
node backend/welcome_mail_server.mjs
```

La sortie doit annoncer le port configure. Pour un test local depuis un deuxieme terminal PowerShell :

```
$body = @{ email = 'votre-email'; firstName = 'Eya' } | ConvertTo-Json -Compress
Invoke-RestMethod -Method Post -Uri 'http://localhost:6500/verification-email' -ContentType 'application/json' -Body $bod
```

3. Connecter l'application Android

L'application Flutter exige une URL HTTPS publique pour appeler le serveur. `localhost` sur l'emulateur Android designe l'emulateur lui-meme et le service local utilise HTTP : une URL HTTPS hebergee ou un tunnel HTTPS est necessaire pour ce parcours depuis l'application.

```
flutter run -d emulator-5554 --dart-define=ACCOUNT_API_URL=https://votre-api.example.com
```

> La cle doit rester dans le fichier backend/.env, jamais dans un message, une capture d'ecran, un commit Git ou une option dart-define. Si une cle a ete partagee, revoquez-la dans Resend et remplacez-la.

En cas d'erreur HTTP, consulter le terminal Node : le serveur indique le code de refus de Resend. Un code 401 indique une cle invalide ou revoquee ; un code 422 signale une requete ou un expediteur a corriger.